Pick Up Where You Left Off: An Efficient Solution for Continuous Vector Similarity Search

Zhuanglin Zheng
SKLCCSE Lab
Beihang University
Beijing, China
zzlin@buaa.edu.cn

Yuxiang Zeng
SKLCCSE Lab
Beihang University
Beijing, China
yxzeng@buaa.edu.cn

Yunzhen Chi
SKLCCSE Lab
Beihang University
Beijing, China
chiyz@buaa.edu.cn

Yongxin Tong
SKLCCSE Lab
Beihang University
Beijing, China
yxtong@buaa.edu.cn

Abstract

Efficient vector similarity search is critical for Retrieval-Augmented Generation (RAG) systems and other real-time AI applications. However, most existing methods are optimized for isolated queries and fail to leverage the continuity and correlation inherent in real-world query streams, such as those in multi-turn dialogues and multi-hop question answering. We formalize this problem as **C**ontinuous **V**ector **S**imilarity **S**earch (**CVSS**). While recent efforts attempt to reuse prior results, they either sacrifice accuracy through semantic caching or yield only marginal efficiency gains. To address this, we propose Reuse, an end-to-end framework that decomposes CVSS into two synergistic components: (1) Reuse Trigger that decides *when to reuse* prior search results, and (2) Reuse Searcher that addresses *how to reuse* them effectively. Together, these components significantly reduce redundant computation while maintaining near-identical recall. Extensive experiments on four real-world datasets show that Reuse achieves 1.6–3.0× higher throughput (QPS) than state-of-the-art methods at the same recall.

CCS Concepts

• **Information systems** → **Nearest-neighbor search**; **Top-k retrieval in databases**.

Keywords

Similarity Search, Nearest Neighbor Search, Vector Database

ACM Reference Format:
Zhuanglin Zheng, Yuxiang Zeng, Yunzhen Chi, and Yongxin Tong. 2026. Pick Up Where You Left Off: An Efficient Solution for Continuous Vector Similarity Search. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817657

Resource Availability:
The source code of this paper has been made publicly available at https://doi.org/10.5281/zenodo.20529235.

1 Introduction

Vector similarity search [17, 18, 24, 25] has become a foundational technique for retrieving semantically relevant information from massive unstructured datasets, such as text and images. By embedding data into a high-dimensional vector space where vector distance reflects semantic similarity, search reduces to finding the nearest neighbors to a query vector. To enable this search at scale, considerable research has focused on effective indexing methods. Among these, graph-based indexes (*e.g.*, HNSW [46], NSG [26]) have emerged as a predominant solution. Consequently, they are widely deployed in production vector databases and search engines.

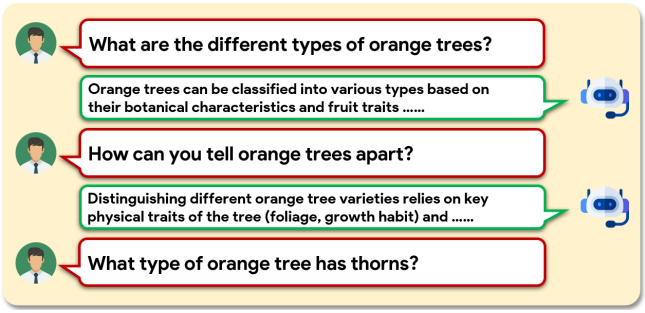


Figure 1: Example of user queries from a real-world dialogue dataset CAsT [22]

Despite extensive research in this area, most studies have focused on optimizing performance for a *single, isolated* query. In real-world applications, however, query workloads typically consist of multiple sequential requests. Modern vector search engines [1, 2, 4, 23] commonly treat each query as an independent snapshot, processing them sequentially without leveraging inter-query relationships. This "*query-as-snapshot*" strategy fails to achieve optimal throughput for correlated queries that arrive in a continuous stream. Such continuous query streams are common in modern AI applications, including multi-hop question answering [47], multi-turn dialogue [8, 65], real-time recommendation [69], and video object tracking [12]. The following example illustrates a typical case.

Example 1. *Fig. 1 shows consecutive dialogue queries from a real-world dataset [22]. These queries are semantically related and revolve around the topic of "orange trees". In a RAG-enabled dialogue system [65], each user query triggers a vector similarity search over an external vector database to retrieve relevant supporting passages. These passages, together with the dialogue history, are then fed into an LLM to generate a coherent and informed response. This pipeline must execute within a strict latency to maintain a real-time conversation. Thus, efficiently processing such continuous query streams is critical.*

Motivation. In such scenarios, queries exhibit *continuous, dynamic*, and *correlated* characteristics. Intuitively, reusing results from a prior query could accelerate the search for subsequent ones. Some

This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817657

recent work [13, 21, 35] has explored semantic caching [11] to exploit this. They typically serve a prior query's result directly as the answer to the current one, which inevitably sacrifices recall. In contrast, TopLoc [51] and SimJoin [64] leverage prior results to guide the search, treating them as initial entry points in the graph index. However, as our theoretical and empirical analysis shows, the improvement offered by them is often marginal. For instance, they reduce latency by at most 18%–23% over the standard query-as-snapshot method on real-world datasets. These limitations highlight a key **technical challenge**: *how to fully leverage query correlation in continuous vector similarity search?*

Our Solution. To address this, we propose Reuse, an end-to-end framework that decomposes the challenge into two interlinked sub-problems: (1) **when to reuse** prior search results to ensure meaningful acceleration, and (2) **how to reuse** them to optimize the graph index traversal. Correspondingly, Reuse integrates two co-designed components: Reuse Trigger and Reuse Searcher. Reuse Trigger addresses the *when-to-reuse* problem by learning a lightweight classifier that predicts whether reusing a prior query's result will reduce the number of distance computations (*i.e.*, the primary latency bottleneck). Reuse Searcher addresses the *how-to-reuse* problem by adaptively pruning redundant neighbors during node expansion in the graph index to accelerate search. Together, these synergistic components achieve substantial computation savings with negligible loss in recall.

Contribution. The main contributions of this work include:

(1) Problem Formulation: We identify the intrinsic limitations of existing vector similarity search solutions in processing sequential, correlated queries, and, in response, are the first to formally define the **C**ontinuous **V**ector **S**imilarity **S**earch (**CVSS**) problem.

(2) Algorithm Design: We propose Reuse, an end-to-end solution with two synergistic components: Reuse Trigger (to decide when to reuse prior query results) and Reuse Searcher (to optimize how to traverse the graph index for vector retrieval).

(3) Experimental Validation: Extensive experiments on four real-world datasets confirm that Reuse achieves 1.6–3.0× higher throughput (QPS) than state-of-the-art methods at identical recall.

Road Map. The rest of this paper is organized as follows. Sec. 2 formally defines the CVSS problem. Sec. 3 introduces our proposed solution, Reuse, followed by its experimental evaluation in Sec. 4. Finally, Sec. 5 reviews related work, and Sec. 6 concludes the paper.

2 Problem Statement

This section first introduces the key concepts and then formally defines the problem. Table 1 summarizes the major notations.

2.1 Basic Concepts

To frame the formal problem, we first introduce two basic concepts: *vector data* and *vector similarity search*.

Definition 1 (Vector Data). *A vector data object (or simply "vector") o is an ordered sequence of d numerical values $(x_1, x_2, \ldots, x_d)$, where d is the data dimension and x_i is its i-th coordinate. A distance function $\mathrm{dis}(o_1, o_2)$ quantifies the dissimilarity between two vectors o_1 and o_2 of the same dimension.*

We denote the vector dataset as O, which consists of n vectors of dimension d. Common metrics for evaluating similarity between vectors include Euclidean distance, inner product, and cosine similarity. Throughout this paper, we treat distance and (dis)similarity as interchangeable concepts and adopt Euclidean distance as the default similarity metric. Notably, our solution is compatible with other distance metrics (see our full paper [3] for evaluations).

Table 1: Summary of frequently used notations

Notation	Description
O	Dataset of n d-dimensional vectors
Q	A sequential online query stream
$\mathcal{S}$	Query answer
o, u, v	A data point
q, k	Query vector, number of nearest neighbors
$\mathrm{dis}(o_1, o_2)$	Distance between two vectors o_1 and o_2
ef	Size of the search candidate list
M	Maximum node degree
ep	Entry point for graph-based index
$\mathcal{W}$	Candidate set for graph-based index

Definition 2 (Vector Similarity Search). *Given a vector dataset O, a query vector q, and a positive integer k, vector similarity search aims to retrieve a subset $\mathcal{S} \subseteq O$ of k nearest vectors to q such that:*

$$|\mathcal{S}| = k \text{ and } \forall v \in \mathcal{S},\ \forall u \in (O \setminus \mathcal{S}),\ \mathrm{dis}(v, q) \leq \mathrm{dis}(u, q) \quad (1)$$

Remark. Because of the curse of dimensionality [55], *exact* vector similarity search in high-dimensional spaces is computationally too expensive to be practical. Consequently, many recent studies [9, 23, 37, 40, 61, 62] have shifted their attention to approximate algorithms. These studies typically evaluate the quality of an approximate algorithm using its *recall*:

$$\text{recall} = \frac{|\mathcal{S} \bigcap \mathcal{S}^*|}{k} \quad (2)$$

where $\mathcal{S}^*$ is the ground-truth and $\mathcal{S}$ is the approximate result set.

2.2 Problem Definition

Traditional vector similarity search is typically regarded as a *query-as-snapshot* operation, where each query is independent of previous searches and has no effect on future ones. Intuitively, we refer to this type of query as **S**napshot **V**ector **S**imilarity **S**earch. In contrast, this work focuses on **C**ontinuous **V**ector **S**imilarity **S**earch (**CVSS**), which we formally define as follows:

Definition 3 (Continuous Vector Similarity Search). *Given a vector dataset O, a stream of online queries $Q = \{q_1, q_2, \ldots, q_m\}$ that arrive sequentially, and a positive integer k, continuous vector similarity search aims to retrieve a subset $\mathcal{S}(q_i) \subseteq O$ of k-Nearest Neighbors (k-NN) for each query $q_i \in Q$ while satisfying:*

- ***Sequential Processing**: The processing of $\mathcal{S}(q_i)$ must be completed before processing for the query q_{i+1} begins.*
- ***k-NN Condition**: Each result set $\mathcal{S}(q_i)$ has k vectors and meets the k-NN condition:*

$$\forall v \in \mathcal{S}(q_i),\ \forall u \in (O \setminus \mathcal{S}(q_i)),\ \mathrm{dis}(v, q_i) \leq \mathrm{dis}(u, q_i) \quad (3)$$

Unlike snapshot vector similarity search, continuous vector similarity search treats each query in the sequence Q not as an isolated query request, but as part of a semantically correlated query

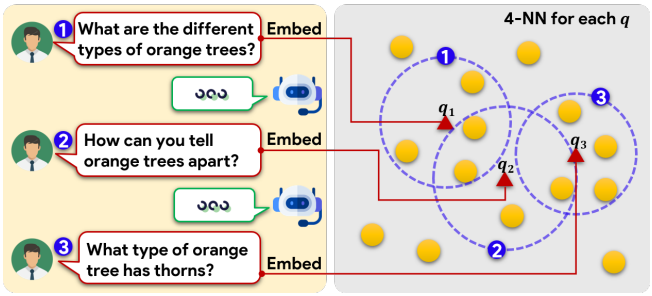


Figure 2: Instances of continuous vector similarity search

stream. This reflects real-world application scenarios where queries are naturally related, such as multi-hop question answering systems [47], LLM-based multi-turn dialogue systems [8, 65], video object tracking [12], and real-time recommendation systems [69]. In these scenarios, user queries exhibit strong semantic relevance and tend to revolve around a set of coherent topics over time.

A core technical challenge in this paradigm lies in how to leverage results from prior queries to accelerate subsequent queries incrementally. This makes continuous vector similarity search fundamentally more challenging than its snapshot-based counterpart. Because exact solutions to a single query are already computationally prohibitive, we focus on designing efficient *approximate algorithms* for CVSS. A concrete example of this problem is illustrated below.

Example 2. *A toy example of the CVSS problem is illustrated in Fig. 2. In a continuous conversational scenario, user queries typically exhibit inherent semantic correlations, ensuring that their corresponding embedding vectors are spatially proximal in the semantic vector space. As a direct consequence, retrieval results for such semantically correlated queries tend to form tight clusters in the semantic space, and may even exhibit partial or complete overlap.*

3 Our End-To-End Solution: Reuse

This section introduces our solution **Reuse** to **C**ontinuous **V**ector **S**imilarity **S**earch (**CVSS**). We first elaborate on the core idea of reusing prior query results in Sec. 3.1 and then break down Reuse into two key components: **Reuse Trigger** which decides when to reuse (Sec. 3.2) and **Reuse Searcher** which optimizes how to reuse (Sec. 3.3). Finally, we describe the integrated solution in Sec. 3.4.

3.1 Motivation

Prevailing solutions to vector similarity search rely on graph-based indexes, with HNSW [46] being the de facto standard. Understanding how these indexes function lays the foundation for our solution. Thus, we first introduce preliminary knowledge of HNSW, and then elaborate on the core idea of our solution Reuse.

3.1.1 Preliminary: Graph-Indexed Vector Similarity Search. HNSW [46] is the state-of-the-art graph-based index that has been widely deployed in vector databases. Its core design is as follows:

- **Hierarchical Structure**: HNSW constructs a multi-layer graph, where each vector corresponds to a node that appears across consecutive layers from bottom to top. Edges in each layer connect similar vectors. Upper layers use sparse graphs for efficient navigation, while the bottom layer uses a dense graph for fine-grained final search.
- **Search Process**: For a similarity query, HNSW starts from a global *entry point* (*ep*) at the topmost layer, greedily traverses to the nearest node, and moves down layer by layer. At each layer, it expands the search by traversing the connected nodes of the current candidate, maintains them in a heap, and ultimately finds the k-NNs.
- **Computational Overhead**: Most of query time is spent on computing vector distances (*e.g.*, 77.2%–87.6% [27]).

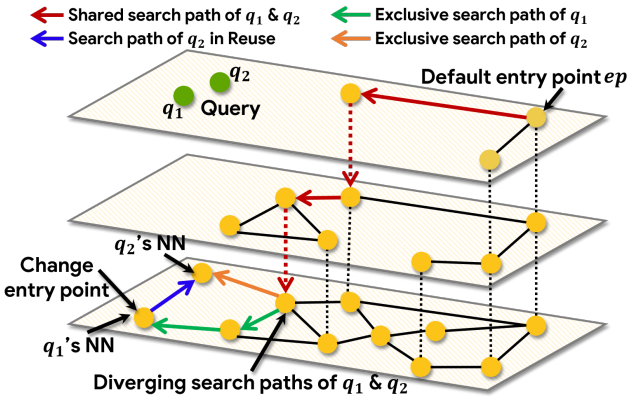


Figure 3: Path of similar queries (q_1, q_2) in HNSW

3.1.2 Observation and Motivation. In real-world applications like multi-hop question answering systems [47] and LLM-based multi-turn dialogues [8, 65], successive queries are often semantically related. This query similarity can lead to substantial overlap in the search regions and traversal paths within HNSW and indicates that reusing prior search progress could accelerate subsequent queries.

As illustrated in Fig. 3, when processing such correlated queries q_1 and q_2 from the real-world multi-turn dialogue dataset [7, 22], their search paths within HNSW (red arrows) exhibit strong overlap. Notably, the search path for q_2 can be significantly shortened by initiating it from the neighbors of q_1's results (blue arrows) instead of restarting from the default entry point.

Motivation. This observation demonstrates that query similarity presents a direct opportunity to reuse prior search results during subsequent traversals, thereby improving search efficiency; realizing this potential requires addressing two *technical challenges*: deciding **when to reuse** previous results (Sec. 3.2) and **how to reuse** them in graph-indexed vector similarity search (Sec. 3.3).

3.2 When To Reuse: Reuse Trigger

The Reuse Trigger component solves the "when to reuse" problem by judging if reusing the prior query's results yields a reduction in search time compared to a standard HNSW search. To make this decision, we design a learning-based binary classifier as follows.

3.2.1 Main Idea. In practice, consecutive queries in CVSS are not always similar. If we misclassify a dissimilar query as similar and reuse some prior query's results for it, the search process will be misled. This introduces unnecessary traversals that increase (rather

Algorithm 1: Training for Reuse Trigger

Input: Vector dataset O, HNSW $\mathcal{G}$ (default entry point ep)
Output: Trained binary classifier (Reuse Trigger)

```
1  Training set Tr ← ∅, vectors Q ← sample from O;
2  foreach sampled query vector q_i ∈ Q do
3      Random entry point v ← sample from O;
4      U_i ← Nodes traversed in HNSW for k-NN search of q_i
         using designated (non-global) entry point v;
5      foreach node (i.e., vector) u_ij ∈ U_i do
6          cnt_1 ← #(distance computation) when searching
             k-NN search of u_ij in HNSW (entry point: q_i's NN);
7          cnt_2 ← #(distance computation) when searching
             k-NN search of u_ij in HNSW (entry point: ep);
8          label ← cnt_1 < cnt_2;        // 1: reuse, 0: not
9          Add tuple (dis(q_i, u_ij), label) into training set Tr;
10 return A binary classifier trained on Tr;
```

than decrease) search time. For example, HNSW's hierarchical structure enables fast and progressive convergence to nearest neighbors. However, initializing a dissimilar query's search from the prior query's bottom-layer neighbors may force the procedure to traverse extra nodes, thereby incurring additional latency.

Intuitively, the fundamental criterion for deciding "when to reuse prior results" is: **whether reusing these results will reduce the main computational overhead**, *i.e.*, the number of vector distance computations, compared to a standard HNSW search.

Guided by this criterion, we model the reuse decision as a *binary classification* task, with labels directly indicating the reuse benefit:

- **Label = 1 (Reusable)**: Reusing a prior query's results as the entry point for the current query requires **fewer distance computations** than a standard HNSW search.
- **Label = 0 (Non-Reusable)**: Reusing the prior query's results requires **more or equal distance computations** than a standard HNSW search, so a standard search is preferred.

3.2.2 Training Classifier in Reuse Trigger. To train the binary classifier, we generate sufficient labeled samples using Alg. 1.

First, we randomly sample vectors from the dataset O to form a query vector set Q. For each $q_i \in Q$, we select a random point v as the initial entry point, perform a k-NN search for q_i, and collect all nodes traversed during this search process into a set $\mathcal{U}_i$ (lines 2–4). Through this procedure, we generate a collection of vector pairs (q_i, u_{ij}) with various distances $\text{dis}(q_i, u_{ij})$, where $u_{ij} \in \mathcal{U}_i$. Here, we use a random point v instead of ep as entry point to expand the sampling range, avoiding confined sampling around ep.

For each vector pair (q_i, u_{ij}), we compute the distance $\text{dis}(q_i, u_{ij})$ and then perform two vector similarity searches for u_{ij}: (1) using q_i's nearest neighbor as the entry point, and (2) using the default entry point ep. We also record the number of vector distance computations as cnt_1 and cnt_2, respectively. We assign a label of 1 if $cnt_1 < cnt_2$ and 0 otherwise, with a weight of $|cnt_1 - cnt_2|$. This creates a training dataset of samples $(\text{dis}(q_i, u_{ij}), \text{label})$ for training a binary classifier.

Table 2: Average number of expanded nodes per vector search

Dataset	SIFT	Laion	GIST	OpenAI
$ef = 20$	22.37	22.05	24.44	24.05
$ef = 100$	100.88	101.12	101.65	102.44
$ef = 500$	500.20	500.46	500.59	501.10

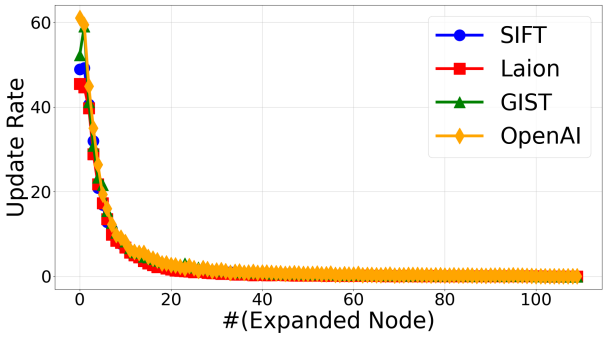


Figure 4: Update rate of the candidate set in bottom-layer search within HNSW on k-NN benchmark datasets

3.3 How To Reuse: Reuse Searcher

To address the "how to reuse" challenge, the Reuse Searcher component introduces an optimization algorithm that leverages the prior query's results to reduce the latency of the current query.

3.3.1 Key Observation: Changing Entry Point Is Not Enough. A straightforward solution [51, 64] is to replace the default entry point with the nearest neighbor of a prior query q_1 that is similar to the current query q_2. While this can avoid traversing the upper layers of HNSW, our analysis shows that it still needs to traverse a large number of nodes in the bottom layer. Since bottom-layer traversal typically constitutes a dominant portion of the total search time [30, 57], we observe that *merely replacing the entry point is inadequate for achieving significant efficiency improvement.*

Specifically, this observation is supported by the theoretical analysis in Lemma 1 and empirical results in Table 2. Lemma 1 shows that even with entry point replacement, searching the bottom layer still requires approximately $ef \times \widetilde{M}$ distance computations (*i.e.*, the dominant factor of search time). To empirically validate this, we measure the number of *expanded nodes* (*i.e.*, nodes whose neighbors are all examined) per query on four widely used k-NN benchmark datasets [5, 58]. The detailed dataset information is provided in Appendix A. As shown in Table 2, this average number closely matches ef. Since each expanded node examines all its $\widetilde{M}$ connected neighbors, the total distance computations align with the theoretical bound $ef \times \widetilde{M}$.

Lemma 1. *During bottom-layer search within HNSW, the number of distance computations scales as $ef \times \widetilde{M}$, where ef is the candidate set size and $\widetilde{M}$ is the average node degree. (Proof in Appendix.)*

3.3.2 Main Idea. The inefficiency identified in Lemma 1 stems from the exhaustive neighbor examination in existing methods. However, when queries q_1 and q_2 are similar, q_1's navigation path

already reveals a high-confidence area around the target. Thus, processing q_2 from q_1's nearest neighbors enables confident pruning of redundant neighbors. Building on this insight, we propose to (1) reuse prior query's results as entry points and (2) access only the top-b most promising neighbors per node, with b tuned adaptively.

This approach requires addressing two **technical challenges**:

(1) Efficient Neighbor Ranking. Computing exact distances for ranking is costly. We adopt Binary Quantization (BQ) [20] to map 32-bit floating-point dimensions to 1-bit representations, enabling fast approximate distance computations via bitwise operations with minimal storage overhead. To further mitigate information loss from uneven dimension importance, we first apply a random rotation to the vector dataset via an orthogonal matrix, which distributes information uniformly across dimensions [34] and reduces quantization error.

(2) Adaptive b Selection. A fixed b either limits speedup (if too large) or harms recall (if too small). Our adaptive strategy for selecting b is based on a key observation formalized below:

> Lemma 2. *During bottom-layer search within HNSW, the update rate of the candidate set decays progressively. (Proof in Appendix.)*

As shown in Fig. 4, the update rate of the candidate set exhibits a clear downward trend across multiple benchmark datasets. This insight implies that the optimal b should correlate with the update frequency f. Thus, we adjust b adaptively from its previous value b_{prev} using Exponential Moving Average [32]:

$$b = (1-\alpha) \times b_{prev} + \alpha \times f + \beta \tag{4}$$

Here, $\alpha \in (0, 1)$ controls how quickly b adapts to changes in f. For example, a larger α gives more weight to recent updates. The integer β ensures a minimum b to prevent overly aggressive pruning.

Remark. Reuse enables confident pruning: the prior query provides high confidence for aggressive pruning while preserving high recall. This synergy, rather than reuse alone, distinguishes our algorithm from existing solutions [51, 64].

3.3.3 Algorithm Details. Alg. 2 presents the optimized algorithm for Reuse Searcher, which consists of two phases: (1) entry point initialization using prior query's results (lines 4–7), and (2) adaptive neighbor pruning during traversal (lines 8–20). It maintains visited nodes in min-heap C and candidate nodes in max-heap $\mathcal{W}$.

(1) Entry Point Initialization. We reuse the k-NN result of a prior query confirmed by Reuse Trigger. Each k-NN vector serves as the entry point at the bottom layer of HNSW. All these vectors are inserted into C and $\mathcal{W}$ (line 6). The upper bound UB of the kth-NN distance to q is updated once $\mathcal{W}$ reaches size ef (line 7).

(2) Adaptive Neighbor Pruning. When expanding the node $v \in C$, we use $\mathcal{B}$ to denote the set of unvisited neighbors of v and sort them by their Binary Quantization [20] distances to the query vector q (lines 9–12). Next, we examine the top-b neighbors: for each neighbor u, compute its true distance ud to q, and insert (u, ud) into the heaps C and $\mathcal{W}$ (lines 14–19). We also update the upper bound UB and track how many times $\mathcal{W}$ has been modified using a counter f. We dynamically adjust b using Eq. (4).

The search terminates when the current node v's distance to q exceeds UB, the upper bound of the kth-NN distance (line 10).

Algorithm 2: Reuse Searcher: Optimized Graph Search

Input: HNSW $\mathcal{G}$, query q, size ef of candidate set
Output: k-Nearest Neighbor (k-NN) to q

1 Min-heap $C \leftarrow$ a set of visited but unexpanded nodes, sorted by distance to q;
2 Max-heap $\mathcal{W} \leftarrow$ a set of candidates sorted by distance to q;
3 $b \leftarrow \widetilde{M}$, upper bound of k-NN distance to q as $UB \leftarrow \infty$;
4 **foreach** *prior query's k-NN* **do** `// Init entry point`
5 Mark ep as an entry point at bottom layer of HNSW $\mathcal{G}$;
6 Insert $(ep, \text{dis}(ep, q))$ into C and $\mathcal{W}$, keep $|\mathcal{W}| \leq ef$;
7 **if** $|\mathcal{W}| = ef$ **then** $UB \leftarrow$ farthest distance to q in $\mathcal{W}$;
8 **while** $C \neq \emptyset$ **do** `// Adaptive Neighbor Pruning`
9 Node v and distance $vd \leftarrow$ extract top tuple in C;
10 **if** $vd \geq UB$ **then break**;
11 Node set $\mathcal{B} \leftarrow$ unvisited neighbors of node v;
12 Sort $\mathcal{B}$ by their binary quantization distances to q;
13 Initialize candidate heap $\mathcal{W}$'s update count $f \leftarrow 0$;
14 **for** $i \leftarrow 1$ **to** b **do** `// Access top-b neighbors`
15 Node $u \leftarrow i$-th node in $\mathcal{B}$, distance $ud \leftarrow \text{dis}(u, q)$;
16 **if** $|\mathcal{W}| < ef$ *or distance* $ud < UB$ **then**
17 Insert (u, ud) into C and $\mathcal{W}$, keep $|\mathcal{W}| \leq ef$;
18 $UB \leftarrow$ farthest distance to q in $\mathcal{W}$;
19 Increase update count $f \leftarrow f + 1$;
20 $b \leftarrow (1-\alpha) \times b + \alpha \times f + \beta$; `// Using Eq. (4)`
21 **return** top-k nearest vectors from $\mathcal{W}$ to query vector q;

3.4 Integrated Framework: Put Them Together

Building on components Reuse Trigger and Reuse Searcher, we present the solution Reuse, whose workflow is illustrated in Fig. 5.

3.4.1 Main Workflow. For each incoming continuous vector similarity search query q_i in the stream Q, we process it in three stages:

(1) Identify Similar Prior Query. Leveraging the inherent semantic relevance of query streams, we maintain a fixed-length sliding window over the stream and identify the most semantically similar query q_* within it. Given the short query streams in our experimental settings, we adopt a linear scan strategy for query localization. For long query streams, we can maintain an (append-only) HNSW index over past queries.

(2) Invoke Reuse Trigger. Next, the binary classifier in Reuse Trigger determines whether reusing q^*'s result will accelerate q_i. If the prediction is negative, q_i is deemed independent and processed via the standard HNSW search.

(3) Perform Reuse Searcher. If Reuse Trigger predicts positive, we apply the optimized graph search algorithm in Reuse Searcher to answer the query q_i using the prior query q^*'s result.

Finally, q_i's k-NN is cached in memory for future reuse.

3.4.2 Theoretical Analysis. We analyze the recall behavior and time complexity of our solution Reuse.

Recall Loss Trend. Lemma 3 shows that the recall loss induced by Reuse Searcher compared to standard HNSW search diminishes

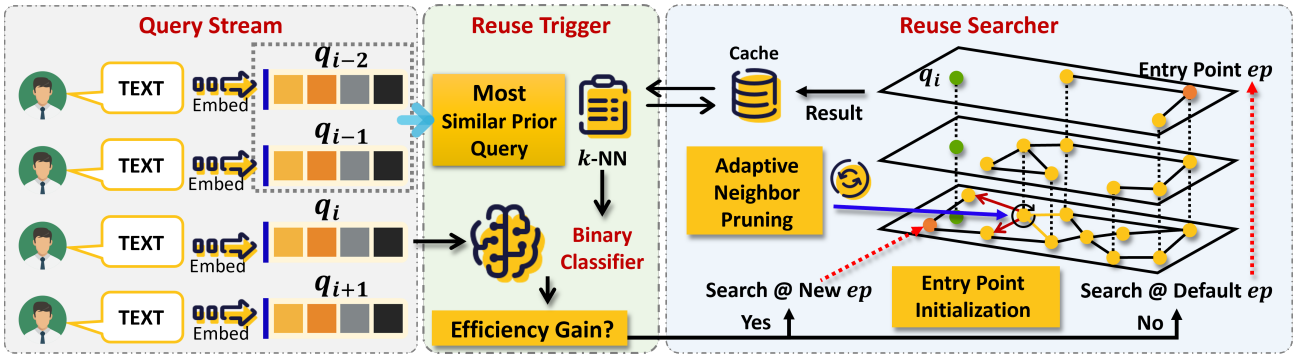


Figure 5: Illustration of overall solution Reuse

Table 3: Statistics of real-world datasets

Dataset	Cardinality	Dimension	Type	Size
ActivityNet	1,640,539	512	Video	3.2 GB
Amazon	1,551,057	768	Text	4.5 GB
CAsT	1,760,180	1024	Text	6.8 GB
QReCC	1,760,180	1024	Text	6.8 GB

rapidly with ef. This suggests a guideline for setting β when updating b in Eq. (4): when ef is sufficiently larger than k, a small β adequately controls recall loss; otherwise, β should be larger to ensure sufficient neighbor expansion.

> LEMMA 3. *The recall loss incurred by Reuse Searcher decays rapidly as the size ef of the candidate set increases. (Proof in Appendix.)*

Complexity Analysis. We mainly analyze time complexity, as Reuse's space cost asymptotically matches HNSW's space cost $O(n)$ [46], plus storage for cached query results. Time complexity differs based on the prediction of Reuse Trigger:

(1) Negative Prediction. Vector similarity search proceeds via the standard HNSW, with $O(\log n)$ time complexity per query.

(2) Positive Prediction. If the prediction is correct, Reuse Searcher reduces distance computations (*i.e.*, the dominant time cost), and hence maintains $O(\log n)$ complexity but with a smaller constant factor. If the prediction is incorrect (*i.e.*, the reused entry point is suboptimal), the search process reduces to single-layer NSW traversal, which requires $O(\log^2 n)$ time [45] in the worst case. Nevertheless, our empirical results demonstrate that Reuse is consistently faster than the standard HNSW search that takes $O(\log n)$ time.

4 Experimental Study

To validate our proposed solution Reuse, we conduct extensive experiments on four real-world datasets. Specifically, we first introduce the experimental setup in Sec. 4.1. Then, we present the evaluation of overall search performance, end-to-end time cost analysis, and ablation study in Sec. 4.2 through Sec. 4.4. Finally, we summarize the key findings in Sec. 4.5.

4.1 Experimental Setup

Dataset. Our experiments are conducted on four real-world datasets in Table 3. Among them, CAsT [22] and QReCC [7] are dialogue datasets that provide conversational utterances and their manually rewritten versions, which we embed as queries over the MSMARCO corpus [52] following prior work on multi-turn dialogue search [38, 41, 65]. ActivityNet [6] consists of video-frame features sampled with a stride of 16, and we form its queries from consecutive segments to emulate video-object-tracking workloads [12]. Amazon [36] provides shopping-session clickstreams over short time windows. We use them as query streams and product metadata as the corpus for embedding, to match the need of real-time recommendation systems [69]. For each dataset, ground-truth is obtained via an exhaustive linear scan over the entire dataset.

In addition to the above four datasets, we further evaluate our solution on **long-running** sessions constructed from the QReCC dataset. We concatenate multiple consecutive dialogues from QReCC to form sessions of approximately 140 queries each. This setup naturally captures the semantic drift that occurs across successive dialogues, reflecting real-world conversational dynamics.

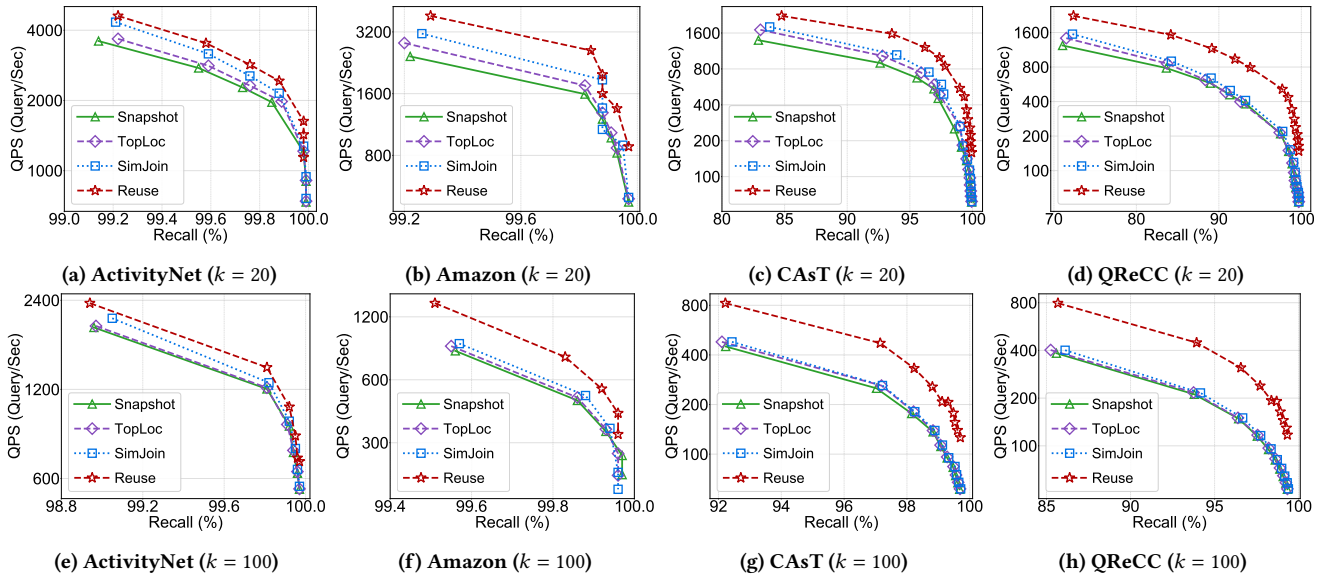
Baselines. We compare our Reuse against three state-of-the-art (SOTA) methods, all of which use HNSW as the underlying index:

- TopLoc [51] adopts the first query's nearest neighbor as the bottom-layer entry point for all subsequent queries.
- SimJoin [64] uses the k-NN of the most recent query as bottom-layer entry points for each incoming query.
- We also evaluate a common solution (named Snapshot) that is used in production vector databases like Milvus [1] and Qdrant [2]. It processes each query independently.

Unlike Reuse, both TopLoc and SimJoin assume reuse always helps (overlooking "when to reuse") and only modify entry points without further pruning neighbors during graph search.

Metrics. We evaluate all algorithms based on two major metrics: (1) *effectiveness* using **recall** (defined in Eq. (2)) and (2) *throughput* in **Queries Per Second (QPS)**. The latency per query includes both processing time and result caching overhead.

Parameter Setting. Following prior work [33, 62], we vary the number of requested neighbors $k \in \{20, 100\}$ and tune the search candidate list size ef from k to 1000 to examine the QPS–recall trade-off. Default values are $k = 20$ and $ef = 200$. For Eq. (4) used in our method, we set the parameters $\alpha = 0.5$ and choose $\beta \in \{0, 1, 2\}$ according to the ratio between ef and k. We adopt a sliding window covering 25% of the average conversation length to identify the most similar prior query, using the Euclidean distance as the default.



(a) **ActivityNet** ($k = 20$) (b) **Amazon** ($k = 20$) (c) **CAsT** ($k = 20$) (d) **QReCC** ($k = 20$)

(e) **ActivityNet** ($k = 100$) (f) **Amazon** ($k = 100$) (g) **CAsT** ($k = 100$) (h) **QReCC** ($k = 100$)

Figure 6: QPS-recall curves of compared methods across four datasets at different k values

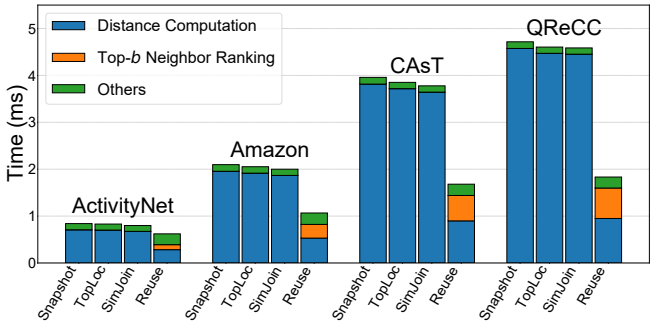


Figure 7: Query latency breakdown

Implementation. All methods are implemented in C++. Snapshot and TopLoc are implemented from scratch by us, while SimJoin uses the authors' provided implementation. Each method is integrated into a high-performance HNSW framework for vector similarity search, HnswLib [46], ensuring a common underlying index. We configure the HNSW index with a maximum node degree $M = 32$ and a construction-time candidate list size $efConstruction = 500$, following the recommended settings in Faiss [23]. We make our code and datasets publicly available on GitHub [3].

Experiment Environment. All the experiments are conducted on a server with an AMD Ryzen 9 5950X CPU and 64 GB RAM, running Ubuntu 24.04.2 LTS.

Due to page limitations, please refer to our online full paper [3] for the hyperparameter study and experiments on other distance metrics (*e.g.*, inner product).

4.2 Overall Search Performance

Fig. 6 shows the QPS-recall curves of all compared methods on four datasets, where methods closer to the upper-right corner exhibit superior performance.

Overall Performance. At any given recall level, our Reuse consistently achieves the highest QPS across all datasets. For example, compared to Snapshot at $k = 20$ and same recall, Reuse improves QPS by up to 1.6, 1.9, 2.7, and 3.0× on ActivityNet, Amazon, CAsT, and QReCC, respectively. As for TopLoc and SimJoin, they improve QPS by up to 1.2 and 1.3× over Snapshot, respectively. Moreover, at a fixed candidate search list ef, the recall loss introduced by Reuse always remains below 0.1%. Sometimes, the recall of Reuse is higher than that of the compared baselines by up to 1.8%.

Comparisons Among Baselines. The QPS advantage of TopLoc and SimJoin over Snapshot diminishes as recall increases. This aligns with our analysis in Sec. 3.3: these methods reduce search time primarily by skipping upper layers of HNSW. However, as recall rises, the time spent on upper-layer search becomes a progressively smaller fraction of the total search time, thereby reducing their relative benefit. For instance, at 99% recall, their QPS over Snapshot is at most 4% on CAsT dataset. In contrast, Reuse reduces search time at both the upper and bottom layers.

Impact of k. Increasing the number k of requested neighbors from 20 to 100 notably diminishes the QPS advantage of TopLoc and SimJoin over Snapshot, as shown in Fig. 6. For example, on QReCC with $k = 100$, their speedup falls below 1.06 and 1.07×, whereas Reuse continues to outperform Snapshot by up to 2.3×. This demonstrates the robustness of Reuse across varying k values.

Long-Running Session. As shown in Fig. 8, Reuse consistently achieves higher QPS than all other compared approaches without any degradation in recall performance. For example, compared with Snapshot, Reuse improves QPS by up to 2.7×, whereas the speedups of TopLoc and SimJoin are below 1.04× and 1.29× respectively. During transitions between consecutive sub-sessions where queries exhibit semantic drift, Reuse Trigger intelligently refrains from reusing stale results initially, then seamlessly switches to reuse as more relevant queries arrive. This result demonstrates that Reuse also maintains superior performance in long-running sessions.

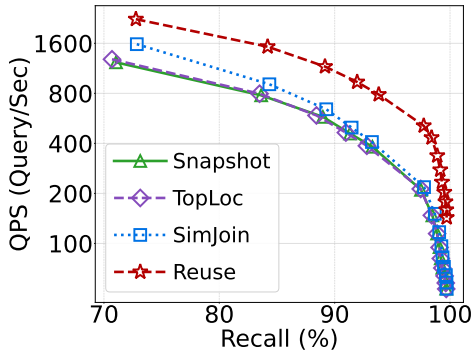
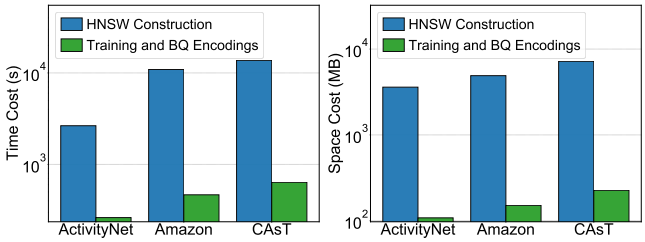


Figure 8: QPS-recall curves of compared methods in long-running sessions



(a) Time overhead **(b) Memory overhead**

Figure 9: Pre-Processing overhead: time and space overhead

4.3 End-to-End Time Cost Analysis

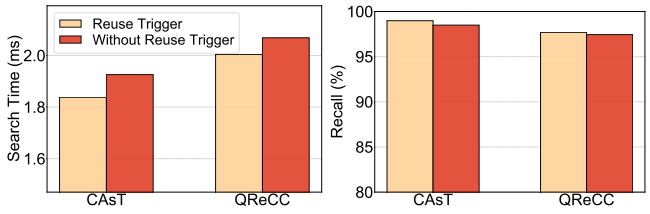
We analyze the time cost of Reuse by evaluating its online query latency (Sec. 4.3.1) and offline pre-processing overhead (Sec. 4.3.2).

4.3.1 Query Latency Breakdown. We decompose query latency into three parts: (1) distance computation, (2) top-b neighbor ranking, and (3) other overheads (*e.g.*, graph traversal), as shown in Fig. 7.

Our method significantly reduces query latency, primarily by lowering distance computation time while adding only minor overhead in top-b neighbor ranking. For example, on QReCC, distance computation time is reduced by 79.2%, whereas it accounts for over 84.3% of the total latency in the baselines. These results confirm that distance computation at the bottom layer remains the primary efficiency bottleneck for the compared baselines, validating our design motivation.

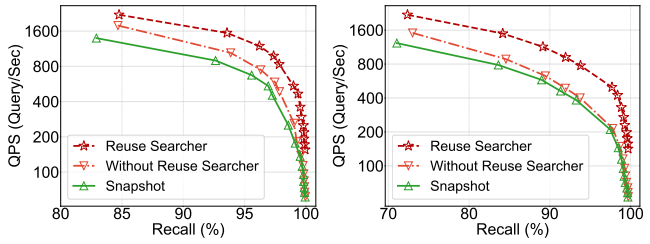
4.3.2 Pre-Processing Overhead. As shown in Fig. 9a, the total pre-processing time of our Reuse consists of two parts: (1) index construction time, and (2) additional overhead for training (Reuse Trigger) and generating binary query (BQ) encodings (Reuse Searcher). The corresponding space costs are presented in Fig. 9b.

The comparison demonstrates that Reuse introduces minimal extra overhead in time and space. The extra pre-processing time accounts for only 4.1%–9.0% of the index construction time across all datasets. Similarly, the extra space cost introduced remains below 3% of the total space cost. These results confirm that the efficiency gains of Reuse are achieved with very low pre-processing overhead.



(a) Search time **(b) Recall**

Figure 10: Result of ablation study on Reuse Trigger



(a) QPS-recall curves on CAsT dataset **(b) QPS-recall curves on QReCC dataset**

Figure 11: Result of ablation study on Reuse Searcher

4.4 Ablation Study

We also conduct ablation studies to evaluate the contributions of Reuse Trigger and Reuse Searcher in our solution Reuse.

4.4.1 Ablation Study on Reuse Trigger. We evaluate the impact of Reuse Trigger by comparing recall and search time of our method with and without it on the CAsT and QReCC datasets. Both versions adopt the same pruning strategy in Reuse Searcher. The query parameter k is set to 20, and the candidate set size ef is set to 200.

As shown in Fig. 10, Reuse Trigger reduces latency and boosts recall on both datasets. The latency improvement comes from accurately identifying similar queries and reusing their results as entry points at the bottom layer of HNSW. The recall gain stems from using multiple entry points instead of a single, fixed entry point. These diverse starting nodes enable the search to explore more graph regions and reduce the risk of being confined to a local neighborhood.

4.4.2 Ablation Study on Reuse Searcher. Fig. 11 evaluates the impact of the component Reuse Searcher in our solution on the CAsT and QReCC datasets.

Integrating Reuse Searcher consistently improves search performance, increasing QPS by up to 2.9× compared to the variant without it. This confirms the effectiveness of our optimized graph search method. Even without Reuse Searcher, Reuse still improves QPS by up to 1.3× over Snapshot on the CAsT dataset. However, this gain diminishes as ef grows (*i.e.*, as recall rises). At 99% recall, the speedup drops below 2%. This validates our claim in Sec. 3.3 that entry point replacement alone is insufficient for high-recall acceleration.

4.5 Summary of Major Findings

Our experimental results highlight three key findings:

- Our method Reuse achieves substantial speedup, improving QPS by up to 1.6–3.0× over the baselines across four datasets. Its recall even surpasses that of Snapshot by up to 1.9% in some cases, and when it slightly lags behind Snapshot, the drop is limited to at most 0.1%. All these improvements come at a marginal pre-processing cost and memory overhead.
- Among baselines, SimJoin often achieves higher QPS than TopLoc and Snapshot, but their query throughput becomes similar at high recall levels.
- Solely adjusting entry points provides limited acceleration, with the benefit diminishing at higher recall targets.

5 Related Work

We review related work from the following two categories: *vector similarity search* and *continuous k-NN search*.

Vector Similarity Search. Vector similarity search supports applications in data mining, information retrieval, and LLM systems [14, 16, 40, 60]. Existing solutions fall into three main categories based on their underlying index structures: graph-based [30, 44, 46], partition-based [10, 49, 66], and hashing-based methods [29, 43]. Among them, HNSW [46] has emerged as a widely adopted graph-based solution and is deployed into production vector databases such as Milvus [1], Weaviate [4], and Qdrant [2]. Orthogonal to these index-centric approaches, Distance Comparison Operation (DCO) based methods [27, 70] accelerate distance computation operations, and can be seamlessly integrated into current vector search frameworks to further improve query latency. However, these methods are primarily designed to optimize *single* query performance.

Prior work on multi-query processing largely focuses on *vector similarity join* [19, 48, 63, 64, 67]. These methods are designed for batched queries that either arrive simultaneously or are collected before execution, unlike continuous queries that arrive sequentially and require real-time responses. To illustrate this gap, we compare against the SOTA method SimJoin [64], and show that adaptations to the CVSS problem result in limited efficiency advantage.

Continuous k-NN Search. Traditional continuous k-NN search leverages temporal coherence in query sequences but is largely confined to low-dimensional spatial datasets [39, 50, 59]. This line of work usually relies on multidimensional data structures [42], such as R-trees [31] and Voronoi diagrams [53], to improve search efficiency. However, these exact solutions do not scale to high-dimensional vector spaces due to the curse of dimensionality [55].

Continuous k-NN search has gained renewed attention in Retrieval-Augmented Generation (RAG) systems [13, 21, 28, 35]. These methods often use semantic caching [11] to accelerate vector retrieval sequences [13, 21, 35]. However, their latency reduction comes at the cost of recall, as they reuse prior query results directly. In contrast, recent work on conversational search [51] preserves recall by using the first query's results as entry points for HNSW search.

Summary. While continuous vector similarity search is already being used in applications like RAG, the problem of reducing latency by fully exploiting query correlation while maintaining recall remains unexplored. This research gap motivates this work.

6 Conclusion

Performing vector similarity searches on a continuous stream is increasingly essential for applications like multi-hop QA and multi-turn dialogue. However, fully leveraging the correlations among sequential queries remains a technical challenge, since existing methods typically treat queries in isolation or rely on simplistic reuse heuristics. To address this gap, we formally define **C**ontinuous **V**ector **S**imilarity **S**earch (**CVSS**) and propose an efficient solution called Reuse. Reuse exploits query continuity and similarity via two synergistic components: (1) a Reuse Trigger that predicts *when* to reuse prior results for acceleration, and (2) a Reuse Searcher that optimizes *how* to traverse the graph index during search. Extensive experiments on four real-world datasets demonstrate Reuse significantly outperforms state-of-the-art methods, achieving a 1.6–3.0× improvement in query throughput at equivalent recall levels.

Acknowledgments

This work was partially supported by the National Natural Science Foundation of China (NSFC) (Grant Nos. 62502023, 62425202, 62336003), the National Key Research and Development Program of China under Grant No. 2023YFF0725103, the Beijing Natural Science Foundation (Z230001), the Fundamental Research Funds for the Central Universities (No. JKF-2025010601998), and the State Key Laboratory of Complex & Critical Software Environment (SKLCCSE). Yuxiang Zeng is the corresponding author.

References

[1] [n. d.]. Milvus. https://github.com/milvus-io/milvus
[2] [n. d.]. Qdrant. https://github.com/qdrant/qdrant
[3] [n. d.]. Source Code, Datasets, and Full Paper. https://github.com/zzlin237/continuous-vector-similarity-search.git
[4] [n. d.]. Weaviate. https://github.com/weaviate/weaviate
[5] 2024. Big ANN Benchmark. https://big-ann-benchmarks.com/
[6] Humam Alwassel, Silvio Giancola, and Bernard Ghanem. 2021. TSP: Temporally-sensitive pretraining of video encoders for localization tasks. In *CVPR Workshops*. 3173–3183.
[7] Raviteja Anantha, Svitlana Vakulenko, Zhucheng Tu, Shayne Longpre, Stephen Pulman, and Srinivas Chappidi. 2021. Open-domain question answering goes conversational via question rewriting. In *NAACL-HLT*. 520–534.
[8] Akari Asai, Sewon Min, Zexuan Zhong, and Danqi Chen. 2023. Retrieval-based Language Models and Applications. In *ACL*. 41–46.
[9] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Inf. Syst.* 87 (2020), 101374.
[10] Artem Babenko and Victor S. Lempitsky. 2015. The Inverted Multi-Index. *IEEE Trans. Pattern Anal. Mach. Intell.* 37, 6 (2015), 1247–1260.
[11] Fu Bang. 2023. Gptcache: An open-source semantic cache for llm applications enabling faster answers and cost savings. In *NLP-OSS*. 212–218.
[12] Nir Ben-Zrihem and Lihi Zelnik-Manor. 2015. Approximate nearest neighbor fields in video. In *CVPR*. 5233–5242.
[13] Shai Aviram Bergman, Ji Zhang, Anne-Marie Kermarrec, Diana Petrescu, Rafael Pires, Mathis Randl, and Martijn de Vos. 2025. Leveraging Approximate Caching for Faster Retrieval-Augmented Generation. In *EuroMLSys*. 66–73.
[14] Gérard Biau and Luc Devroye. 2015. *Lectures on the nearest neighbor method*. Vol. 246. Springer.
[15] Prosenjit Bose and Pat Morin. 2004. Online Routing in Triangulations. *SIAM J. Comput.* 33, 4 (2004), 937–951.
[16] Sebastian Bruch. 2024. *Foundations of Vector Retrieval*. Springer.
[17] Matteo Ceccarello, Alexandra Levchenko, Ioana Ileana, and Themis Palpanas. 2025. Evaluating and Generating Query Workloads for High Dimensional Vector Similarity Search. In *SIGKDD*. 5299–5310.
[18] Chao Chen, Tian Zhou, Yanjun Zhao, Hui Liu, Rong Jin, and Liang Sun. 2025. Does Vector Quantization Fail in Spatio-Temporal Forecasting? Exploring a Differentiable Sparse Soft-Vector Quantization Approach. In *SIGKDD*. 143–154.
[19] Yanqi Chen, Xiao Yan, Alexandra Meliou, and Eric Lo. 2025. DiskJoin: Large-scale Vector Similarity Join with SSD. *Proc. ACM Manag. Data* 3, 6 (2025), 1–27.

[20] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. 2015. BinaryConnect: Training Deep Neural Networks with binary weights during propagations. In *NeurIPS*. 3123–3131.

[21] Camille Couturier, Spyros Mastorakis, Haiying Shen, Saravan Rajmohan, and Victor Rühle. 2025. Semantic Caching of Contextual Summaries for Efficient Question-Answering with Language Models. *CoRR* abs/2505.11271 (2025).

[22] Jeffrey Dalton, Chenyan Xiong, Vaibhav Kumar, and Jamie Callan. 2020. CAsT-19: A Dataset for Conversational Information Seeking. In *SIGIR*. 1985–1988.

[23] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Trans. Big Data* 12, 2 (2026), 346–361.

[24] Zeheng Fan, Yuxiang Zeng, Zhuanglin Zheng, and Yongxin Tong. 2025. FedVSE: A Privacy-Preserving and Efficient Vector Search Engine for Federated Databases. *PVLDB* 18, 12 (2025), 5371–5374.

[25] Zeheng Fan, Yuxiang Zeng, Zhuanglin Zheng, Binhan Yang, and Yongxin Tong. 2025. FedVS: Towards Federated Vector Similarity Search with Filters. In *SIGKDD*. 579–590.

[26] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *PVLDB* 12, 5 (2019), 461–474.

[27] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proc. ACM Manag. Data* 1, 2 (2023), 137:1–137:27.

[28] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *CoRR* abs/2312.10997 (2023).

[29] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *VLDB*. 518–529.

[30] Zengyang Gong, Yuxiang Zeng, and Lei Chen. 2025. Accelerating Approximate Nearest Neighbor Search in Hierarchical Graphs: Efficient Level Navigation with Shortcuts. *PVLDB* 18, 10 (2025), 3518–3530.

[31] Antonin Guttman. 1984. R-Trees: A Dynamic Index Structure for Spatial Searching. In *SIGMOD*. 47–57.

[32] Charles C. Holt. 2004. Forecasting seasonals and trends by exponentially weighted moving averages. *International Journal of Forecasting* 20, 1 (2004), 5–10.

[33] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *NeurIPS* 32 (2019).

[34] Hervé Jégou, Matthijs Douze, Cordelia Schmid, and Patrick Pérez. 2010. Aggregating local descriptors into a compact image representation. In *CVPR*. 3304–3311.

[35] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2026. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *ACM Trans. Comput. Syst.* 44, 1 (2026), 2:1–2:27.

[36] Wei Jin, Haitao Mao, Zheng Li, Haoming Jiang, Chen Luo, Hongzhi Wen, Haoyu Han, Hanqing Lu, Zhengyang Wang, Ruirui Li, Zhen Li, Monica Xiao Cheng, Rahul Goutam, Haiyang Zhang, Karthik Subbian, Suhang Wang, Yizhou Sun, Jiliang Tang, Bing Yin, and Xianfeng Tang. 2023. Amazon-M2: A Multilingual Multi-locale Shopping Session Dataset for Recommendation and Text Generation. (2023).

[37] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* 7, 3 (2021), 535–547.

[38] Yannis Katsis, Sara Rosenthal, Kshitij Fadnis, Chulaka Gunasekara, Young-Suk Lee, Lucian Popa, Vraj Shah, Huaiyu Zhu, Danish Contractor, and Marina Danilevsky. 2025. mtRAG: A Multi-Turn Conversational Benchmark for Evaluating Retrieval-Augmented Generation Systems. *Trans. Assoc. Comput. Linguistics* 13 (2025), 784–808.

[39] Chuanwen Li, Yu Gu, Jianzhong Qi, Ge Yu, Rui Zhang, and Wang Yi. 2014. Processing Moving kNN Queries Using Influential Neighbor Sets. *PVLDB* 8, 2 (2014), 113–124.

[40] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data - Experiments, Analyses, and Improvement. *IEEE Trans. Knowl. Data Eng.* 32, 8 (2020), 1475–1488.

[41] Xiaoyu Li, Xiao Li, Li Gao, Yiding Liu, Xiaoyang Wang, Shuaiqiang Wang, Junfeng Wang, and Dawei Yin. 2025. Proactive Guidance of Multi-Turn Conversation in Industrial Search. (2025), 706–717.

[42] Qiyu Liu, Maocheng Li, Yuxiang Zeng, Yanyan Shen, and Lei Chen. 2025. How good are multi-dimensional learned indexes? An experimental survey. *VLDB J.* 34, 2 (2025), 17.

[43] Wanqi Liu, Hanchen Wang, Ying Zhang, Wei Wang, Lu Qin, and Xuemin Lin. 2021. EI-LSH: An early-termination driven I/O efficient incremental c-approximate nearest neighbor search. *VLDB J.* 30, 2 (2021), 215–235.

[44] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: Hierarchical Graph Structure Based on Voronoi Diagrams for Solving Approximate Nearest Neighbor Search. *PVLDB* 15, 2 (2021), 246–258.

[45] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Inf. Syst.* 45 (2014), 61–68.

[46] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.

[47] Vaibhav Mavi, Anubhav Jangra, and Adam Jatowt. 2022. A Survey on Multi-hop Question Answering and Generation. *CoRR* abs/2204.09140 (2022).

[48] Ahmed Metwally and Michael Shum. 2024. Similarity Joins of Sparse Features. In *SIGMOD*. 80–92.

[49] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Umar Farooq Minhas, Jeffrey Pound, Cédric Renggli, Nima Reyhani, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2024. Incremental IVF Index Maintenance for Streaming Vector Search. *CoRR* abs/2411.00970 (2024).

[50] Kyriakos Mouratidis, Man Lung Yiu, Dimitris Papadias, and Nikos Mamoulis. 2006. Continuous Nearest Neighbor Monitoring in Road Networks. In *VLDB*. 43–54.

[51] Cristina Ioana Muntean, Franco Maria Nardini, Raffaele Perego, Guido Rocchietti, and Cosimo Rulli. 2025. Efficient Conversational Search via Topical Locality in Dense Retrieval. In *SIGIR*. 2749–2753.

[52] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. In *NeurIPS Workshop*, Vol. 1773.

[53] Atsuyuki Okabe, Barry Boots, Kokichi Sugihara, Sung Nok Chiu, and David G. Kendall. 2000. *Spatial Tessellations: Concepts and Applications of Voronoi Diagrams, Second Edition.* Wiley.

[54] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proc. ACM Manag. Data* 1, 1 (2023), 54:1–54:27.

[55] Petra Perner. 2002. Data Mining - Concepts and Techniques. *Künstliche Intell.* 16, 1 (2002), 77.

[56] Milos Radovanovic, Alexandros Nanopoulos, and Mirjana Ivanovic. 2010. Hubs in Space: Popular Nearest Neighbors in High-Dimensional Data. *J. Mach. Learn. Res.* 11 (2010), 2487–2531.

[57] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *NeurIPS*.

[58] Christoph Schuhmann, Richard Vencu, Romain Beaumont, Robert Kaczmarczyk, Clayton Mullis, Aarush Katta, Theo Coombes, Jenia Jitsev, and Aran Komatsuzaki. 2021. LAION-400M: Open Dataset of CLIP-Filtered 400 Million Image-Text Pairs. *CoRR* abs/2111.02114 (2021).

[59] Yufei Tao, Dimitris Papadias, and Qiongmao Shen. 2002. Continuous Nearest Neighbor Search. In *VLDB*. 287–298.

[60] Jianguo Wang, Eric Hanson, Guoliang Li, Yannis Papakonstantinou, Harsha Simhadri, and Charles Xie. 2024. Vector Databases: What's Really New and What's Next? *PVLDB* 17, 12 (2024), 4505–4506.

[61] Jingdong Wang, Ting Zhang, Jingkuan Song, Nicu Sebe, and Heng Tao Shen. 2018. A Survey on Learning to Hash. *IEEE Trans. Pattern Anal. Mach. Intell.* 40, 4 (2018), 769–790.

[62] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *PVLDB* 14, 11 (2021), 1964–1978.

[63] Yifan Wang, Vyom Pathak, and Daisy Zhe Wang. 2024. Xling: A Learned Filter Framework for Accelerating High-Dimensional Approximate Similarity Join. *CoRR* abs/2402.13397 (2024).

[64] Jiadong Xie, Jeffrey Xu Yu, and Yingfan Liu. 2025. Fast Approximate Similarity Join in Vector Databases. *Proc. ACM Manag. Data* 3, 3 (2025), 158:1–158:26.

[65] Zihao Yi, Jiarui Ouyang, Zhe Xu, Yuwen Liu, Tianhao Liao, Haohao Luo, and Ying Shen. 2026. A Survey on Recent Advances in LLM-Based Multi-turn Dialogue Systems. *ACM Comput. Surv.* 58, 6 (2026), 148:1–148:38.

[66] Yuxiang Zeng, Yongxin Tong, and Lei Chen. 2023. LiteHST: A Tree Embedding based Method for Similarity Search. *Proc. ACM Manag. Data* 1, 1 (2023), 35:1–35:26.

[67] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *OSDI*. 377–395.

[68] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *PVLDB* 16, 8 (2023), 1979–1991.

[69] Yan Zhao, Shoujin Wang, Yan Wang, and Hongwei Liu. 2023. MbSRS: A multi-behavior streaming recommender system. *Inf. Sci.* 631 (2023), 145–163.

[70] Zhuanglin Zheng, Yuxiang Zeng, Chenchen Liu, Yunzhen Chi, Binhan Yang, and Yongxin Tong. 2026. Distance Comparison Operations Are Not Silver Bullets in Vector Similarity Search: A Benchmark Study on Their Merits and Limits. In *42nd IEEE International Conference on Data Engineering, ICDE 2026, Montreal, Canada, May 4-8, 2026*. 1–14.

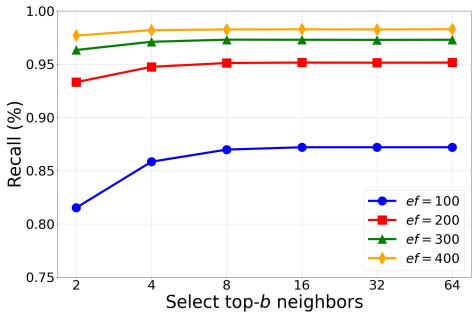


Figure 12: Impact of b on GIST dataset when $k = 100$

Algorithm 3: Bottom-layer search within HNSW

Input: HNSW $\mathcal{G}$, query q, size ef of candidate set
Output: k-Nearest Neighbor (k-NN) to q

1 Min-heap $C \leftarrow$ a set of visited but unexpanded nodes, sorted by distance to q;
2 Max-heap $\mathcal{W} \leftarrow$ a set of candidates sorted by distance to q;
3 Node $ep \leftarrow$ entry point determined by upper-layer search;
4 Insert $(ep, \text{dis}(ep, q))$ into C and $\mathcal{W}$;
5 Upper bound of k-NN distance to q as $UB \leftarrow \infty$;
6 **while** C *is not empty* **do**
7 Node v and distance $vd \leftarrow$ extract top tuple in C;
8 **if** $vd \geq UB$ **then break** ;
9 **foreach** $u \in v$*'s neighbors in* $\mathcal{G}$ **do**
10 **if** u *has not been visited* **then**
11 Distance $ud \leftarrow \text{dis}(u, q)$;
12 **if** $|\mathcal{W}| < ef$ *or distance* $ud < UB$ **then**
13 Insert (u, ud) into C and $\mathcal{W}$;
14 Maintain $|\mathcal{W}| \leq ef$;
15 $UB \leftarrow$ farthest distance to q in $\mathcal{W}$;
16 **return** top-k nearest vectors from $\mathcal{W}$ to query vector q;

A Vector Datasets Used in Section 3

The vector datasets [5, 58] used in Section 3 are detailed in Table 4. These datasets have large-scale vectors with dimensions ranging from 128 to 1536 and comprise diverse raw data types, including image, text, and multimodal data. Moreover, these datasets have been commonly used to evaluate the graph-based index algorithms for vector similarity search [40, 44, 54, 62, 68]. Their use thus provides robust empirical validation for our lemmas.

Table 4: Statistics of the benchmark vector datasets used in Section 3

Dataset	Dim.	Raw Data	Cardinality	Size (GB)
SIFT	128	Image	1,000,000	0.5
Laion	512	Multimodal	1,000,448	2.0
GIST	960	Image	1,000,000	3.6
OpenAI	1536	Text	2,321,096	14

B Deferred Proofs in Section 3

This section presents the proofs deferred from Section 3.

B.1 Standard Algorithm for Vector Similarity Search within HNSW

The search algorithm in HNSW performs a Best-First Search (BFS) over the hierarchical graph $\mathcal{G}$. It takes the nearest neighbor of the upper-layer graph as the entry point for the lower-layer graph, and in the search process for each layer, this procedure maintains two heaps: a min-heap C containing visited but unexpanded nodes sorted by their distance to q, and a max-heap $\mathcal{W}$ that dynamically stores the current top-ef nearest candidates to q. Additionally, upper bound UB denotes the farthest distance to q in the heap $\mathcal{W}$.

Starting from the entry point ep within this layer, the algorithm iteratively extracts the closest node v from C. If its distance vd exceeds the current upper bound UB, the search terminates early, as no remaining node can improve the k-NN result. Otherwise, each unvisited neighbor u of v will be examined. If $\mathcal{W}$ is not yet full or u is closer than UB, u is inserted into both C (for possible future expansion) and $\mathcal{W}$ (for updating candidate results). The size of $\mathcal{W}$ is kept at most ef by discarding the farthest element when necessary, and the upper bound of k-th NN distance UB is updated accordingly. Finally, the top-k closest vectors to q are returned from the candidate set $\mathcal{W}$.

B.2 Proof of Lemma 1

Lemma 1. *During bottom-layer search within HNSW, the number of distance computations scales as $ef \times \widetilde{M}$, where ef is the candidate set size and $\widetilde{M}$ is the average node degree.*

Proof. Alg. 3 shows the detailed procedure of bottom-layer traversal within HNSW. Based on this algorithm, we can establish the following prerequisites:

- A node is *expanded* when all its connected neighbors have been explored and their distances to query q have been computed in line 9.
- Distance computations occur exclusively during node expansions. Thus, the number of expanded nodes directly determines the number of distance computations (*i.e.*, the efficiency bottleneck).

When the algorithm terminates, ef nodes in the candidate set $\mathcal{W}$ have been expanded (line 8). Thus, there are at least ef expanded nodes. Since HNSW approximates a Delaunay graph [46], there exists a monotonic path between any two nodes where each step moves closer to the target [15]. Thus, any additional expanded nodes (beyond those in $\mathcal{W}$) occur when navigating from distant regions toward q. In practice, the entry point ep_0 at the bottom layer is typically close to q with high probability [26], so few such long-distance navigations are required. Consequently, the total number of expanded nodes rarely exceeds ef significantly.

Each expanded node triggers $\widetilde{M}$ distance computations, where $\widetilde{M}$ is the average node degree at the bottom layer. Therefore, the total number of distance computations approximates $ef \times \widetilde{M}$. □

B.3 Proof of Lemma 2

Lemma 2. *During bottom-layer search within HNSW, the update rate of the candidate set $\mathcal{W}$ decays progressively.*

Proof. As specified in Alg. 3, the upper bound derived after t updates to the candidate set $\mathcal{W}$ (in lines 13-14) is denoted $UB^{(t)}$, with $\mathcal{U}^{(t)}$ representing the set of unvisited points at this stage.

Since $\mathcal{W}$ maintains the ef nearest points to the current query q, once $|\mathcal{W}| = ef$, each update to $\mathcal{W}$ replaces its farthest candidate node with one whose distance to q is less than the current $UB^{(t)}$. This ensures that $UB^{(t)}$ is non-increasing after every update:

$$UB^{(t+1)} \leq UB^{(t)} \tag{5}$$

Meanwhile, the set of unvisited nodes $\mathcal{U}^{(t)}$ contracts continuously as the search expands nodes. Consequently, the subset of potential results satisfying $\mathrm{dis}(u, q) < UB^{(t)}$ shrinks correspondingly over the search process. This subset is formally defined as:

$$S^{(t)} = \left\{ u \in \mathcal{U}^{(t)} \mid \mathrm{dis}(u, q) < UB^{(t)} \right\} \tag{6}$$

In this definition, $|S^{(t)}|$ diminishes with increasing t.

Therefore, the condition in line 12 grows increasingly difficult to satisfy as the search proceeds. Accordingly, the probability of finding a node that meets the insertion condition diminishes with ongoing search traversal, which implies that the update rate of $\mathcal{W}$ declines progressively during bottom-layer traversal. □

B.4 Proof of Lemma 3

Lemma 3. *The recall loss incurred by Reuse Searcher decays rapidly as the size ef of the candidate set increases.*

Proof. Let $\mathcal{N}^*(q)$ denote the ground-truth top-k nearest neighbors to the query vector q. Consider a node $p \in \mathcal{N}^*(q)$ that is initially missed because it does not rank among the top-b neighbors of the first few nodes expanded.

Since p is a true k-NN of q, it is likely to be a "hub node" [56] in the high-dimensional vector space (*i.e.*, it appears in the nearest-neighbor lists of many other data objects). In the static HNSW index $\mathcal{G}$, this hub property translates into a high in-degree: many nodes have p among their top-b nearest neighbors (stored in their pre-computed candidate lists $\mathcal{B}$).

As the search continues and expands more candidates (increasing ef), it will eventually visit some of these nodes that have p in their top-b neighbor lists. When such a node is expanded, p will be discovered and added to the min-heap C (since it improves the top-ef k-NN results).

Therefore, the probability that p remains unvisited after examining ef candidates decays rapidly as ef grows. The more candidates we expand (larger ef), the higher the chance we expand a node that is connected to the hub p, where p is likely to be recovered. □

This theoretical result of recall loss is validated by our experimental results. As shown in Fig. 12, recall increases with the rise of b and subsequently plateaus at a saturation point. For curves corresponding to larger ef, a smaller b is sufficient to attain saturated recall.